

Como o app reconhece uma configuração de mão

Uma explicação do método de classificação, sua base matemática e a decisão de trocar o Random Forest no piloto web.

Distinção importante: o MediaPipe continua rastreando a mão e produzindo 21 landmarks. O que foi substituído é o classificador que interpreta esses pontos e escolhe a letra.

1. O que significa “aproximar coordenadas”?

O nome técnico mais próximo é classificação por vizinho mais próximo (nearest neighbor), aplicada aos exemplos de landmarks. Cada pose vira um vetor numérico; o sistema compara o vetor da webcam com exemplos rotulados e escolhe a classe cujo exemplo mais próximo tenha a menor distância.

A ideia é clássica em reconhecimento de padrões: Fix e Hodges apresentaram trabalhos iniciais sobre a regra nos anos 1950; Cover e Hart publicaram uma análise teórica influente em 1967. O projeto usa essa família de método, mas sua normalização, pesos e regra de escore são escolhas específicas desta implementação.

2. Fluxo do reconhecimento estático

ETAPA	O QUE ACONTECE
1 · Câmera e rastreo	MediaPipe no navegador localiza 21 pontos da mão, cada um com coordenadas x, y e z estimado.
2 · Vetor	As três coordenadas de cada ponto formam até 63 números que descrevem a pose observada.
3 · Normalização	O pulso vira a origem e as coordenadas são ajustadas pela proporção da imagem e pelo tamanho da palma.
4 · Comparação	A pose atual é comparada com exemplos conhecidos; vence a classe com menor distância, desde que o escore e a estabilidade temporal sejam suficientes.

As letras J, X e Z são especiais: como envolvem movimento, seguem um caminho separado que analisa a trajetória em vários frames.

3. Como a comparação é feita

Normalização. O ponto do pulso é subtraído dos demais pontos para a posição da mão na imagem não dominar a comparação. Depois, os valores são divididos por uma medida da palma para que uma mão maior ou mais próxima da câmera não seja automaticamente considerada diferente.

Variantes de orientação. Antes de comparar, o classificador testa a pose original e pequenas rotações, além da reflexão horizontal. Isso tolera alguma diferença de inclinação e lateralidade. Para cada exemplo, é considerada a menor distância entre essas variantes.

Distância ponderada. A implementação usa uma raiz da média dos erros quadráticos, com pesos. As pontas dos dedos recebem peso maior; o eixo z recebe peso menor porque a profundidade estimada por uma câmera comum é menos estável.

$$d(a,b) = \sqrt{\sum_i w_i (a_i - b_i)^2 / \sum_i w_i}$$

Em que a é a mão da webcam, b é um exemplo conhecido e w_i é o peso daquela coordenada. A distância é pequena quando as poses normalizadas são parecidas.

Escolha e ambiguidade. O algoritmo guarda a menor distância por classe. Se U for a mais próxima, mas R estiver quase empatada, a margem entre as duas é pequena e o escore é reduzido. Assim, o sistema pode recusar uma pose ambígua em vez de sempre aprovar o candidato menos distante.

```
similaridade = exp(-0.5 * (d1 / 0.28)^2)
margem = limitar((d2 - d1) / 0.12, entre 0 e 1)
escore = min(similaridade, 0.5 + 0.5 * margem)
```

O escore exibido é semelhança geométrica, não uma probabilidade estatística calibrada. A engine exige escore maior que 85% por um segundo contínuo. A confirmação de um segundo é uma proteção temporal separada da distância: frames ruins, perda da mão ou interrupções zeram a confirmação.

Base matemática

Conceito	Uso no projeto
Vetores e distância	Representar a pose e medir diferença entre landmarks.
Normalização	Reduzir efeito de posição, escala e proporção da imagem.
Erro quadrático ponderado	Dar pesos diferentes a dedos e eixos; produzir a distância.
Minimização discreta	Selecionar a menor distância entre as classes disponíveis.
Transformação por rotação/reflexão	Comparar variantes com pequenas diferenças de orientação.

4. Por que o Random Forest foi substituído?

O modelo original foi treinado em Python com Scikit-learn e salvo como artefato Joblib/pickle. O navegador não executa esse arquivo diretamente. Seria possível exportar ou reimplementar as árvores em JavaScript, mas isso exigiria um conversor e a garantia de que a extração e a normalização das coordenadas fossem idênticas às do Python.

A auditoria também encontrou limitações no material disponível: 137 registros no CSV, 20 letras com somente uma amostra, ausência de numerais e classes E, T e U ausentes no modelo pickle. Para A, o modelo antigo apresentou aproximadamente 0,63 em uma referência, abaixo do limiar de 0,85 usado pela mecânica do jogo. O código antigo da webcam chamava `predict` e não aplicava confirmação temporal.

Random Forest original	Comparação atual por exemplos
Aprende regras em árvores durante o treinamento.	Armazena exemplos e compara a pose com cada um durante a predição.
Artefato Joblib/pickle ligado ao ambiente Python.	Implementado em JavaScript e executado no navegador.
Modelo disponível incompleto para o alfabeto e frágil diante do dataset pequeno.	Mais fácil de inspecionar e ajustar com os exemplos que existem.
Poderia voltar a ser opção com exportação e dados melhores.	É uma linha de base; não há evidência de que seja universalmente mais preciso.

5. O que a mudança não resolve

- Poucos exemplos não descrevem a variação de mãos, pessoas, câmeras, ângulos e iluminação.
- Se um dedo cobre outro, o MediaPipe pode estimar pontos parecidos para sinais diferentes. Um classificador que só recebe esses pontos não recupera necessariamente o cruzamento que a imagem escondeu.
- R e U podem ser geometricamente próximos. A comparação explícita essa ambiguidade, mas não garante que sempre consiga separá-los.
- O eixo z é uma estimativa relativa da profundidade, não uma medição 3D absoluta.

A escolha foi pragmática: uma base mais transparente, corrigível e compatível com o piloto web. Ela não foi escolhida após um benchmark que provasse maior acurácia que o Random Forest.

Próxima avaliação técnica

Com mais dados e teste separado por pessoa, comparar protótipos/vizinhos, Random Forest, SVM e MLP sobre landmarks; para sinais com oclusão, testar também imagem RGB ou uma fusão de imagem e landmarks. Separar participantes entre treino e teste evita que capturas quase iguais apareçam nos dois grupos.

Uma explicação curta para apresentar

“O MediaPipe continua rastreando a mão. O que mudou foi o classificador: o Random Forest original estava em formato Python/Joblib, que o navegador não executa diretamente, e o modelo disponível tinha limitações no dataset. Para o piloto web, implementei uma comparação por vizinho mais próximo: normalizo os landmarks, comparo a pose com exemplos conhecidos e reduzo o escore se duas letras ficarem muito próximas. Essa escolha facilita executar e inspecionar o sistema no navegador; ela é uma linha de base, não uma prova de maior precisão. Com mais dados, quero comparar os métodos em um teste separado por pessoa.”

Referências e implementação

Cover, T. M.; Hart, P. E. “Nearest Neighbor Pattern Classification”, IEEE Transactions on Information Theory, 1967. DOI: 10.1109/TIT.1967.1053964. doi.org/10.1109/TIT.1967.1053964

Fix e Hodges. Referência histórica à regra de vizinho mais próximo nos relatórios técnicos de 1951, apresentada em revisão acadêmica: Probabilistic Nearest Neighbors Classification.

Scikit-learn. Guia oficial de métodos de vizinhos mais próximos: Nearest Neighbors.

No projeto: ``js/classifier.js`` implementa a normalização e a distância; ``js/vision.js`` escolhe o classificador estático e direciona J/X/Z para análise de trajetória; ``treinar.py`` documenta o treinamento Random Forest do protótipo original.

Material preparado para explicar a arquitetura atual do piloto. A classificação geométrica avalia padrões de landmarks e não certifica correção linguística ou fluência em LIBRAS.